

Acid Fabric

Plataforma white-label de inteligência de tendências. Motor proprietário, licenciado por tenant.

Marco: Fase 2 — Tenancy + RLS concluída

Produto: Acid Fabric (base técnica: trends-agent)

Versão de aplicação: 0.1.0 · **Schema:** migration 0016 · **Postgres:** 17

Data: 15 de julho de 2026 · **Confidencial**

01. Sumário executivo

O que é o produto, hoje, em uma página.

O Acid Fabric é um produto de inteligência de tendências culturais. Ele coleta sinais reais de múltiplas fontes sociais e de imprensa, filtra o ruído com uma metodologia proprietária de curadoria, e entrega dois artefatos: um **radar** contínuo de oportunidades por marca e **relatórios visuais** compartilháveis por link, sem PDF nem envio de arquivo.

A base técnica nasceu como ferramenta interna single-tenant da agência Caramelo. Esta versão marca a virada para **plataforma white-label multi-tenant**: o mesmo motor de inteligência, agora licenciável para terceiros (estúdios, agências, holdings, empresas), com isolamento de dados por tenant, identidade e controle central da ACID. O motor de inteligência permanece proprietário e não customizável, é ele o produto que está sendo licenciado.

5	16	3	1024
FONTES DE SINAL	MIGRATIONS APLICADAS	CAMADAS DE PROMPT	DIMS POR VETOR

Estado desta versão: WI-0 (fonte única de marca), Fase 1 (tenancy) e Fase 2 (RLS estrito + identidade por JWT) concluídas e em produção. Os motores existentes (radar, mapa semântico, vetores, geração de report) seguem intactos, a virada foi 100% aditiva.

02. Visão de produto

De ferramenta de uma agência para plataforma licenciada.

Antes (single-tenant)

Um app dedicado a uma agência. Conhecimento de marca hardcoded no código, sem isolamento de dados, sem noção de "cliente da plataforma". Escala impossível sem duplicar o sistema.

Agora (Acid Fabric)

Cada **tenant** (o licenciado) tem branding próprio, perfil criativo próprio e monitora as marcas dos seus clientes. A ACID controla chaves, custo, créditos, planos e módulos de forma central. Dados isolados por tenant no banco.

Terminologia travada

TERMO	SIGNIFICADO	NO SCHEMA
tenant	O cliente da ACID: estúdio, agência, holding ou empresa. O "tipo" é campo, não muda o schema.	tenants.tipo
marca	A marca que um tenant monitora (o cliente dele). O DNA da marca é subido pelo próprio tenant.	marcas
perfil criativo	A lente de leitura do tenant. Bounded: não sobrescreve o motor.	tenants.perfil_criativo

Hierarquia: um tenant tem um perfil criativo e monitora N marcas. Folga futura já prevista no schema: holding com sub-contas via `parent_tenant_id`.

03. O motor de inteligência (o IP)

O diferencial proprietário. É o que está sendo licenciado, e o que nenhum tenant edita.

Arquitetura de prompt em três camadas

CAMADA	O QUE É	GOVERNANÇA
1. Motor ACID	Metodologia macro: curadoria de planner, "rejeite o óbvio", framing de analista, sinais fracos vs fortes, trava anti-fabricação, busca adjacente.	Travada. Propriedade da ACID. Nenhum tenant edita.
2. Perfil do tenant	A lente criativa do licenciado. Ponto de vista de leitura.	Customizável, mas bounded. Não sobrescreve a camada 1.
3. DNA da marca	Produto, tom, perfil comportamental, universos culturais, o que evitar, termos de busca.	Fonte única (<code>marcas.yaml_l_conhecimento</code>), usada por radar E report.

Dois pilares que definem a qualidade

Busca adjacente (o entorno)

Em vez de só procurar menções diretas à marca, o motor varre o entorno cultural onde a audiência já vive (interesse/contexto), estilo interest targeting. Há uma lane cultural em português, uma em inglês para early signals globais, e a lane de marca. É o que separa "monitorar hashtag" de "ler cultura".

Anti-fabricação

Regra dura: nunca gerar tendência ou fato sem dado raspado real por trás. A trava anti-fabricação vive no relatório, não na busca. Cada insight se ancora em sinais coletados, com links de fontes. Sem dado, sem drop.

04. Capacidades atuais

O que a plataforma faz hoje, em produção.

Radar de tendências

Varredura contínua e agendável por marca (`intervalo_horas`). Coleta multi-fonte, dedup semântico contra memória vetorial, scoring de hype em três eixos (densidade, transbordo, velocidade) e classificação de status (em alta / subindo / estabilizando / esfriando). Gera "drops": oportunidades com fato, gancho de produto e fontes. Roda de forma autônoma via GitHub Actions com `service_role` .

Geração de relatório

A partir de um briefing situacional (e, opcionalmente, de uma marca cadastrada), o motor deriva termos de busca, coleta sinais, e produz um relatório visual estruturado: tendências, oportunidades, sugestões de copy, radar de sinais, insights e glossário. Quando há marca, a busca soma os termos evergreen do DNA aos pontuais do briefing.

Mapa semântico

Visualização em grafo (React Flow) das relações entre sinais e temas de uma marca, construída sobre os embeddings vetoriais. Módulo isolado do caminho de report.

Compartilhamento e colaboração

Fluxo de rascunho, revisão e publicação. Report publicado vira link público lido por qualquer pessoa sem conta (cliente ou time), sem exportar arquivo. Trilha de auditoria completa (quem mudou o quê e quando) e painel de administração de usuários.

Coleta multi-fonte: Instagram, TikTok, Twitter/X, Google News e Reddit via Apify. LinkedIn é a única fonte ligável por marca (para casos B2B). Embeddings via Voyage (vetor de 1024 dimensões, índice HNSW no pgvector).

05. Multi-tenancy e segurança

A fronteira de isolamento real entre tenants. O trabalho central desta versão.

Identidade por JWT

A associação usuário → tenant vive em `tenant_users`. Um **Custom Access Token Hook** do Supabase Auth injeta os claims `tenant_id` e `tenant_role` dentro do token a cada emissão. As policies de RLS leem o tenant direto do token, sem tocar no banco a cada linha.

```
# toda emissão de token (login / refresh)
usuário loga
  ↓ Supabase Auth chama custom_access_token_hook(event)
  ↓ função busca o tenant_id em tenant_users
  ↓ injeta { tenant_id, tenant_role } no token
  ↓ token assinado → RLS já sabe quem é o tenant
```

Row-Level Security estrito

Todas as tabelas com dado de tenant carregam a policy `tenant_isolation`: só enxerga/escreve a linha cujo `tenant_id` casa com o claim do token, ou se for super-admin ACID. A sequência de aplicação foi desenhada para nunca trancar acesso:

- Backfill de tudo para o tenant inicial (zero órfãos).
- Policy transitória com escape, inerte até o hook ligar.
- Hook ligado e claim validado para todos os usuários.
- Aperto: remoção do escape, isolamento passa a estrito.

Papéis

PAPEL	ESCOPO
Super-admin ACID (<code>acid_admins</code>)	Staff da ACID. Enxerga todos os tenants. Opera o produto.
Admin de tenant (<code>tenant_users.role</code>)	Administra usuários e dados do próprio tenant.
Editor / Viewer	Operação e leitura dentro do tenant.
Anônimo	Lê apenas relatórios com status publicado (o link de compartilhamento).

Isolamento validado em produção (transação com rollback contra o schema real): usuário com claim vê apenas o próprio tenant; usuário sem claim vê zero; anônimo vê apenas publicados; super-admin vê tudo. A geração autônoma (GitHub Actions, `service_role`) contorna RLS por design, então coleta e geração seguem intactas.

06. Modelo de dados

Tabelas principais e seu papel.

TABELA	PAPEL	RLS
tenants	O licenciado. Tipo, status, branding, perfil criativo, seats, cobrança.	isolamento
tenant_users	Vínculo usuário↔tenant com papel (admin/editor/viewer).	isolamento + self-read
acid_admins	Super-admin da ACID, sobre todos os tenants.	leitura própria
marcas	Marca monitorada. <code>yaml_conhecimento</code> é a fonte única de DNA.	isolamento
reports	Relatórios gerados. Ligados a marca e tenant; status controla a publicação.	tenant + anon(publicado)
trends_radar	Drops do radar por marca (insight, hype, ganchos, fontes).	isolamento
radar_runs	Histórico de execuções do radar (sinais, drops, modelo, status).	isolamento
radar_raw_data	Sinais brutos + <code>embedding</code> vector(1024) para dedup e mapa.	isolamento
report_audit	Trilha de auditoria de toda mudança em relatório.	leitura autenticada

As tabelas de radar denormalizam `tenant_id` (além de pendurar em `marca_id`) para evitar join dentro da policy de RLS. Decisão de performance.

07. Arquitetura técnica

Aplicação Next.js 14 (App Router) + TypeScript. Tailwind CSS. React 18. React Flow para o grafo. Framer Motion. Deploy em Vercel.	Dados e auth Supabase: Postgres 17, Auth (email/senha + custom claims via hook), Storage para imagens, RLS como fronteira de isolamento. pgvector (HNSW) para busca semântica.
Inteligência Anthropic API (Claude Sonnet) para curadoria e geração. Voyage para embeddings (1024 dims). Prompt em três camadas com trava anti-fabricação.	Coleta Apify actors: Instagram, TikTok, Twitter/X, Google News, Reddit, LinkedIn. Execução autônoma do radar via GitHub Actions (service_role).

Superfície de código atual: 16 migrations, ~24 componentes de UI, 9 rotas de app + APIs, módulo de radar isolado em `lib/radar/*`, mapa semântico em `lib/canvas/buildGraph.ts`.

08. Estado das fases

Onde estamos no plano de virada.

FASE	ESCOPO	STATUS
WI-0	Fonte única de marca. Report passa a ler o DNA do YAML da marca; conhecimento hardcoded neutralizado; seleção de marca na criação.	CONCLUÍDA
Fase 1	Camada de tenancy. Tabelas <code>tenants / tenant_users</code> , <code>tenant_id</code> aditivo em todas as tabelas de dado, backfill do tenant inicial.	CONCLUÍDA
Fase 2	RLS + identidade. Claim <code>tenant_id</code> no JWT via hook, policies de isolamento estrito, super-admin ACID.	CONCLUÍDA
Fase 3	Créditos e metering. Ledger de créditos, débito nos entypoints de custo, enforcement por plano.	PRÓXIMA
Fase 4	Console master ACID + painel do cliente + branding dinâmico. Módulos, assinaturas, seats, cobrança.	PLANEJADA
Fase 5	Fabric Lake vetorial. Data lake cross-tenant de inteligência derivada e de-identificada.	PLANEJADA

Restrição dura respeitada: nenhuma mudança quebrou os motores existentes. A virada foi aditiva (coluna nullable → backfill → constraint), com portão de verificação por fase (build + lint + smoke test) e RLS sequenciado para nunca trancar acesso.

Histórico de schema (marcos)

MIGRATIONS	MARCO
0001 – 0008	Reports, fluxo async, revisão/publicação, auditoria, admins.
0009 – 0012	Módulo de radar: marcas, drops, memória vetorial, runs, métricas.
0013	WI-0: <code>reports.marca_id</code> (fonte única de marca).
0014	Fase 1: schema de tenancy + backfill.
0015 – 0016	Fase 2: RLS transitória e depois isolamento estrito.

Acid Fabric — Documento técnico da versão. ACID Tech. Confidencial. Gerado em 15/07/2026. Base técnica: projeto trends-agent (Supabase quwivphvrugbjksprks). Este documento descreve o estado em produção no marco da Fase 2.